

高科幣平台完整架構文件

校園 GPU × XRP Ledger — AI 服務 + 算力出租平台
版本 v1.0 | 競賽規劃用

目錄

- 1. 專案概覽
- 2. 整體系統架構
- 3. 算力量化機制 (CU 系統)
- 4. XRP Ledger 特性應用總覽
- 5. Payment Channel 微支付機制
- 6. 高科幣 (GKC) 發行機制
- 7. 高科幣 vs 直接用 XRP
- 8. 代幣經濟模型
- 9. MVP 產品規劃
- 10. 競賽論述重點

1. 專案概覽

背景

校園引入多台大型 GPU Server，目前用於提供學生使用本地部署模型。以此為基礎，結合 XRP Ledger 的金融原語，建立一個開放的 AI 服務與算力出租平台。

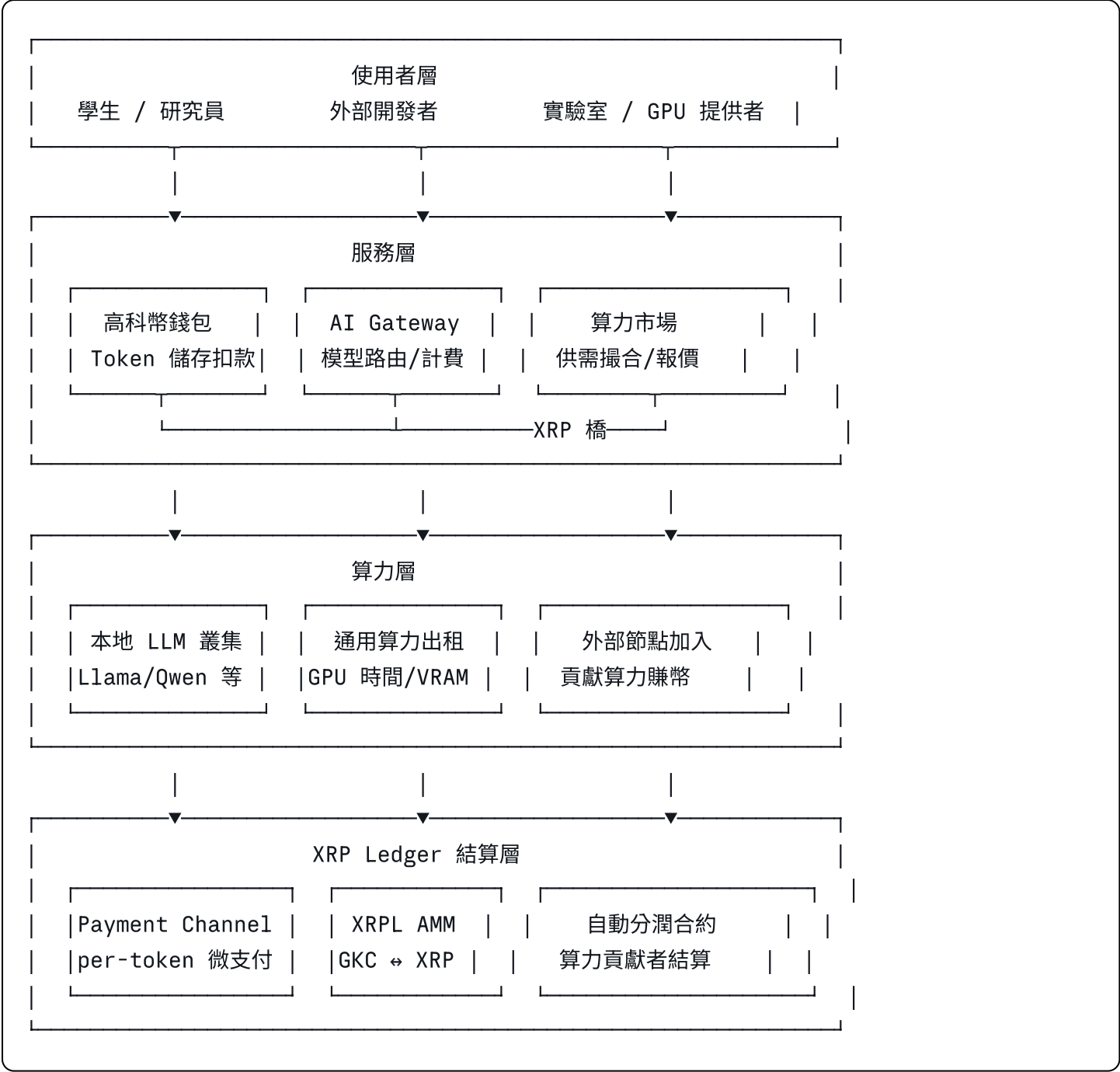
核心概念

- 使用 AI 服務 → 消耗高科幣 (GKC)，按 token 數即時扣費
- 貢獻算力 → 賺取高科幣 (GKC)，按算力單位 (CU) 計算報酬
- 高科幣 → 可在 XRPL AMM 池與 XRP 互換，形成閉環經濟

三條產品線

產品線	對象	計費單位	結算方式
AI 推論服務	學生、研究員、外部開發者	Token 數	Payment Channel
算力出租	需要訓練/微調的研究者	GPU 時間 / CU·小時	Escrow
算力貢獻	各實驗室、外部節點	CU 貢獻量	Hooks 自動分潤

2. 整體系統架構



3. 算力量化機制（CU 系統）

問題背景

各實驗室硬體規格不同（GPU 型號、VRAM、頻寬各異），需要一個統一單位讓所有節點可以公平比較與定價。

算力單位（Compute Unit, CU）

CU 是平台內部的抽象計量單位，由四個維度加權計算：

$$CU = (FP16\text{算力得分} \times W_1) + (VRAM\text{容量得分} \times W_2) + (\text{記憶體頻寬得分} \times W_3) + (\text{實測吞吐量得分} \times W_4)$$

各維度分數以平台最高規格機器為基準 = 100 分，其他機器按比例換算。

預設權重配置

維度	預設權重	說明
FP16 算力 (TFLOPS)	40%	推論速度上限
VRAM 容量 (GB)	25%	可載入模型的大小上限
記憶體頻寬 (GB/s)	20%	Token 生成速度 (LLM 為 memory-bound)
實測吞吐量 (tokens/s)	15%	反映真實環境表現，防止規格虛標

權重可由平台治理投票調整，例如純算力出租場景可提高 FP16 權重；LLM 推論場景可提高頻寬權重。

常見 GPU 參考 CU (預設權重)

GPU 型號	FP16 (TFLOPS)	VRAM (GB)	頻寬 (GB/s)	參考 CU	Tier
A100 80G	312	80	2000	~100	S
RTX 4090	165	24	1008	~62	S
RTX 3090	71	24	936	~48	A
RTX 4080	97	16	717	~46	A
RTX 3080	45	10	760	~34	B
T4 (推論)	65	16	300	~32	B
RTX 3070	41	8	448	~27	B
RTX 2080Ti	27	11	616	~24	C

節點認證流程

1. 申請加入：提交硬體規格文件

2. 自動 Benchmark：平台執行標準測試腳本 (Llama 7B token throughput + FP16 矩陣乘法)

3. CU 認定：測試結果上傳 XRPL 作為鏈上憑證

4. 定期複測：每日自動重跑，實測分數低於申報值 80% 觸發警示

5. 自動下架：連續三次不達標，從活躍節點名單移除

定價公式

每小時費用（GKC）= 基礎幣值 × 節點CU × 稀缺係數

稀缺係數（Scarcity Factor）：

- 供不應求時 > 1（自動漲價）
- 供過於求時 < 1（讓資源不浪費）
- 由 XRPL Hooks 鏈上自動計算，無需人工介入

4. XRP Ledger 特性應用總覽

本平台共使用 7 個 XRPL 原生特性，全部無需部署外部智能合約。

特性對照表

XRPL 特性	應用場景	解決的問題	對應交易類型
Payment Channel	AI 推論按 token 計費	高頻微支付不能每筆上鏈	PaymentChannelCreate/Claim/Fund
IOU / Issued Currency	高科幣（GKC）發行	需要平台自訂代幣作為計費單位	TrustSet + Payment
AMM	GKC ↔ XRP 兌換池	代幣需要與 XRP 之間的流動性	AMMCreate/Deposit/Withdraw
Escrow	算力任務保證金	先付錢還是先做事的信任問題	EscrowCreate/Finish/Cancel
Hooks	自動分潤給算力貢獻者	多節點按 CU 比例結算需要自動化	帳戶層 WebAssembly 邏輯
DEX（訂單簿）	GKC 市場交易	進階用戶需要限價單交易	OfferCreate/Cancel
Path Finding	跨幣種自動路由	降低門檻，用任意幣種付款	ripple_path_find RPC

為什麼選 XRP Ledger 而非 Ethereum

比較項目	XRPL	Ethereum
Payment Channel	原生支援	需要部署 State Channel 合約

比較項目	XRPL	Ethereum
代幣發行	IOU 原生，零部署成本	需要 ERC-20 合約 + 審計
AMM	原生內建（2024）	需要 Uniswap 等外部協議
Escrow	原生支援 Crypto-condition	需要智能合約
結算時間	3-5 秒	12-15 秒（PoS 後）
手續費	~0.00001 XRP（固定極低）	隨網路壅塞浮動
智能合約需求	0 個	5 個以上

5. Payment Channel 微支付機制

核心概念

「先在鏈上開一個保險箱，之後的收據都在鏈下交換，最後才一次結帳。」

三個階段

階段一：開通通道（On-chain，一次）

平台呼叫 PaymentChannelCreate：

- 鎖定一定量 XRP 作為預付金
- 設定到期時間（建議 24 小時）
- 指定收款方（使用者錢包地址）

→ 產生唯一 Channel ID

階段二：鏈下收據交換（Off-chain，每次推論）

每次 AI 推論後：

1. 計算本次 token 數
2. 更新累計 XRP 金額
3. 用平台私鑰簽名
4. 收據附在 API response 回傳

收據結構 (Claim)：

```
{
  channel_id: "aB3f...",      // 通道唯一識別碼
  amount: 210000,             // 累計 drops (0.21 XRP)，只增不減
  signature: "Ed25519..."   // 平台私鑰簽名，防偽
}
```

使用者只需保存最新一張收據，舊的可以丟棄。

階段三：結算關閉（On-chain, 一次）

使用者提交 `PaymentChannelClaim`：

- Ledger 驗證簽名
- 自動分配：使用者收到累計金額，餘額退還平台

特殊情況：

- 使用者不提交 → 通道到期後平台自行關閉取回餘額
- 雙方都有保障，無需信任對方

效益比較

方式	100 次推論的鏈上交易數	手續費	延遲
每次直接上鏈	100 筆	$100 \times 0.00001 \text{ XRP}$	3-5 秒/次
Payment Channel	2 筆（開+關）	$2 \times 0.00001 \text{ XRP}$	< 10ms/次

實作注意事項

- 收據 `amount` 不得超過通道鎖定的 **XRP** 總量，後端需檢查剩餘額度
- 剩餘額度低於閾值時，提醒使用者充值或自動開新通道
- 使用 `xrpl.js` 的 `signPaymentChannelClaim(channelId, amount, privateKey)` 簽名
- 每日最多一條活躍通道，超過後自動合併結算

6. 高科幣（GKC）發行機制

帳戶架構

發行者帳戶（冷錢包）

Issuer Account

- 離線保管
- `DefaultRipple = true`
- 極少動用

營運帳戶（熱錢包）

→授權→ Operational Account

- 日常鑄幣 / 轉帳
- 線上運行
- 如被攻擊不影響發行根權限

`DefaultRipple = true` 必須在帳戶初始化時設定，一旦開啟無法關閉，此設定讓代幣可以在持有者之間自由流通。

發行流程

步驟一：使用者建立信任線（TrustSet）

```
javascript
```

```
// 使用者註冊時 SDK 自動執行，使用者無感
{
  TransactionType: "TrustSet",
  Account: "使用者錢包地址",
  LimitAmount: {
    currency: "GKC",
    issuer: "平台發行者地址",
    value: "10000000" // 信任上限，不影響發行能力
  }
}
```

步驟二：鑄幣（Payment from Issuer）

```
javascript
```

```
// 使用者充值 XRP 後，平台即時鑄造對應 GKC
{
  TransactionType: "Payment",
  Account: "發行者地址",
  Destination: "使用者地址",
  Amount: {
    currency: "GKC",
    issuer: "發行者地址",
    value: "500" // 鑄造 500 GKC
  }
}
// 發行者帳戶餘額變為負數 = 已發出去的量，這是 XRPL IOU 的正常設計
```

步驟三：銷毀（Payment back to Issuer）

```
javascript
```

```
// 使用者消費 GKC 後，平台將 GKC 送回發行者帳戶 = 銷毀
// 鏈上有完整的發行與銷毀記錄
```

AMM 流動性池建立

```
javascript
```

```
// 建立 GKC / XRP 交易對
{
  TransactionType: "AMMCreate",
  Asset: { currency: "XRP" },
  Asset2: { currency: "GKC", issuer: "發行者地址" },
  Amount: "100000000000", // 注入 10,000 XRP
  Amount2: { value: "1000000", currency: "GKC", issuer: "..." },
  TradingFee: 300 // 0.3% 手續費
}
```

7. 高科幣 vs 直接用 XRP

核心分工

XRP 解決「怎麼付錢」，GKC 解決「付多少錢」。XRP 是結算層（真錢），高科幣是應用層（點數）。

詳細比較

比較面向	高科幣（GKC）	直接用 XRP
定價穩定性	平台自訂匯率，使用者成本可預期	隨市價浮動，使用者無法預期
算力激勵	可憑空發幣獎勵貢獻者，無需動用 XRP 儲備	需要預備足夠 XRP 才能發放獎勵
平台商業空間	AMM 手續費流回平台池；可發優惠券、加碼	手續費直接燒掉，平台拿不到
生態綁定	持幣用戶有動機留在平台	用戶隨時可跳槽
開發複雜度	中等（需維護 Trust Line、AMM、鑄毀邏輯）	最簡單（只需 Payment Channel）
法規風險	自行發幣可能需要申報	使用既有幣種，風險最低

雙層架構的優勢

使用者感知層：GKC（穩定定價、激勵設計、商業空間）

↓ AMM 兌換

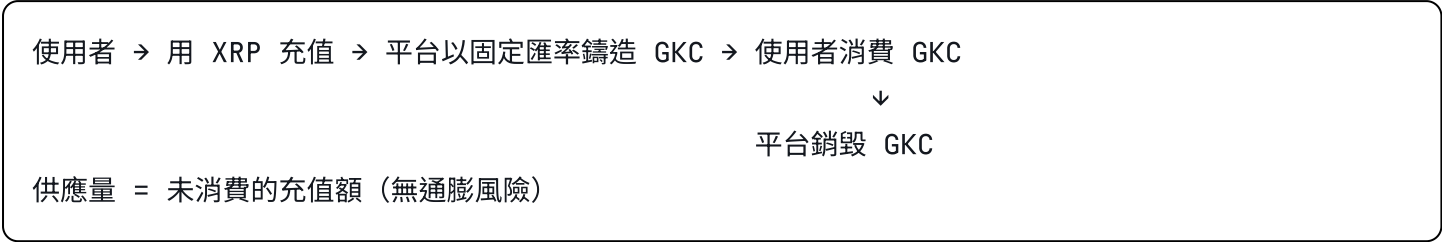
底層結算層：XRP（快速結算、低手續費、全球流通）

這個架構讓定價層和結算層解耦。未來若要改用 RLUSD 穩定幣結算，不需要動到 GKC 的計費邏輯。

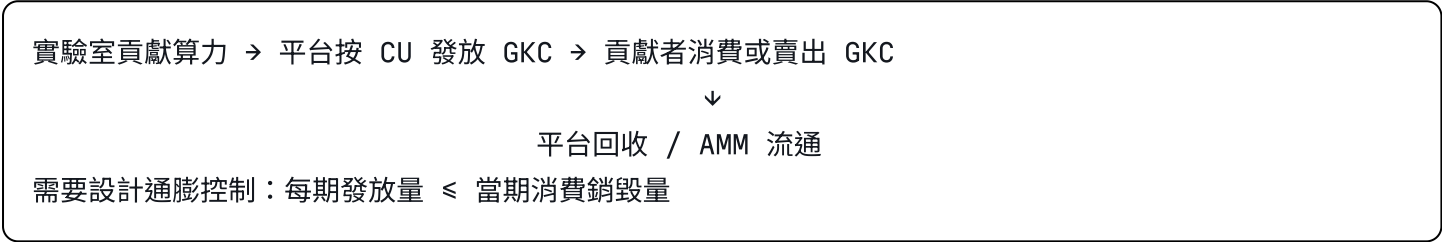
8. 代幣經濟模型

雙軌發幣模型

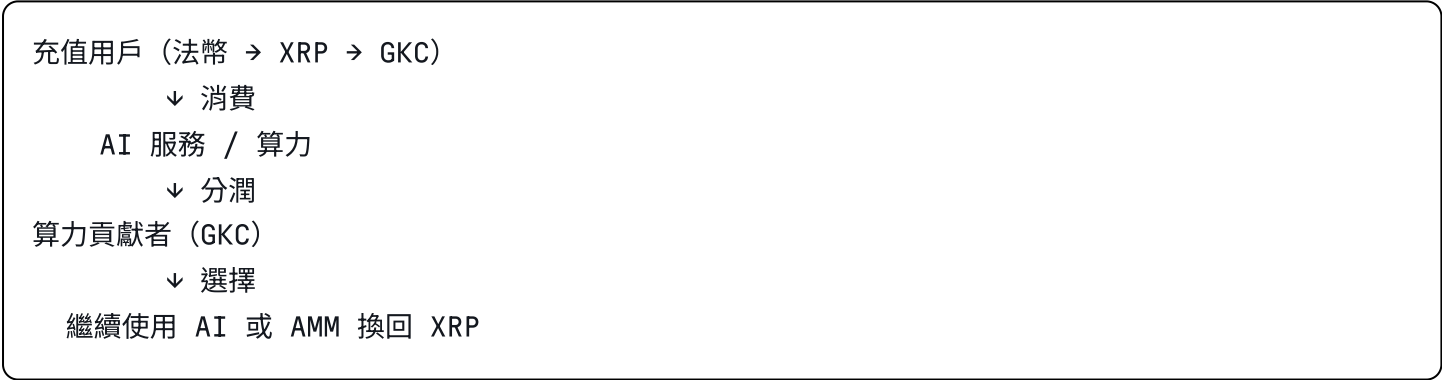
模型 A：充值型（穩定幣邏輯）—— 適用 AI 服務側



模型 B：挖礦型（激勵邏輯）—— 適用算力貢獻側



雙軌閉環



GKC 兌換率設計建議

服務類型	計費單位	建議初始匯率
LLM 推論	per 1K tokens	1 GKC
圖像生成	per image	5 GKC
算力出租	per CU·小時	10 GKC
算力貢獻獎勵	per CU·小時	8 GKC（略低於租用價，平台賺差額）

9. MVP 產品規劃

Demo 場景一：AI 推論 + 自動扣款

目標：展示 Payment Channel 計費鏈路完整跑通

流程：

1. 使用者呼叫 LLM API (例如 /v1/chat/completions)
2. 後端計算 input + output token 數
3. 換算 GKC 金額，更新 Payment Channel 收據
4. 前端即時顯示：剩餘 GKC、累計 token 數、鏈上 tx hash

展示重點：

- GKC 餘額即時扣減（毫秒級）
- 推論結束後才結算，不影響回應速度
- 每筆收據有 Ed25519 簽名，可在 XRPL Explorer 驗證

Demo 場景二：高科幣充值（XRP → GKC）

目標：展示 XRPL AMM + IOU 發幣閉環

流程：

1. 使用者掃 QR code 或連接錢包
2. 選擇充值金額，系統顯示即時 GKC 匯率
3. 確認後：AMM swap (XRP → GKC) + TrustSet (自動)
4. 前端顯示 GKC 餘額更新 + 鏈上交易記錄

展示重點：

- 使用者不需要了解 Trust Line 機制，全程無感
- XRPL Testnet 真實交易，評審可在瀏覽器驗證

Demo 場景三：算力貢獻者自動分潤

目標：展示 Hooks 自動結算邏輯

流程：

1. 模擬一個外部節點完成算力任務（跑完一批推論）
2. 平台驗算完成 → 觸發 Hooks 邏輯
3. XRP 自動按 CU 比例分配給各貢獻節點
4. 儀表板顯示各節點的即時收益

展示重點：

- 全程無人工介入
- 鏈上可查每筆分潤記錄
- 多節點同時結算（展示可擴展性）

技術選型建議

元件	建議方案	說明
LLM 推論	Ollama / vLLM	快速起服務，API 相容 OpenAI 格式
後端	Node.js + xrpl.js	xrpl.js SDK 有完整範例
前端	React + Xaman SDK	Xaman 是 XRPL 主流錢包，有 Web SDK
測試網路	XRPL Testnet	免費領測試幣，Hooks Testnet 也有
鏈上瀏覽器	XRPL Explorer	xrpl.org/explorer，評審可即時驗證

開發優先順序

Week 1：XRPL 帳戶設定 + GKC 發幣 + Trust Line 自動化
Week 2：Payment Channel 完整鏈路（開 → 簽 → 關）
Week 3：AI Gateway 整合計費邏輯 + 前端 Dashboard
Week 4：AMM 流動性池 + 算力節點 CU Benchmark 腳本
Week 5：Hooks 分潤邏輯 + Demo 腳本彩排

10. 競賽論述重點

為什麼選 XRP Ledger

- 金融原語完備**：Payment Channel、Escrow、AMM、DEX 全部原生支援，不需要部署任何智能合約，安全性更高，開發成本更低。
- 高頻微支付場景的唯一解**：AI 推論計費需要毫秒級回應、接近零手續費。XRPL Payment Channel 是目前公鏈中最適合這個場景的解決方案。
- 雙層架構符合真實商業邏輯**：GKC 作為應用層計量單位，XRP 作為結算層，與 AWS 等雲服務的定價邏輯一致，可被主流市場理解與接受。
- 校園資源最大化利用**：把閒置 GPU 時間商品化，不僅服務內部師生，也可開放給外部研究者，形成真實的市場需求。

預期評審問題與回答

Q：為什麼不直接用 XRP 而要發 GKC？ A：XRP 是市場定價的資產，價格波動讓服務成本無法預期。GKC 是平台內部的計量單位，讓定價與算力資源直接掛鉤，同時保留激勵設計空間。兩者分工不同，缺一不可。

Q：為什麼不用以太坊或 Solana？ A：以太坊達到同等功能需要部署 5 個以上合約，gas 費用與高頻支付場景不相容。Solana 雖然快，但缺乏 Payment Channel 原生支援，且 AMM 需要外部協議。XRPL 是把這些金融原語燒進共識層的，最適合本場景。

Q：Hooks 還在 Testnet，這個功能算數嗎？ A：Demo 展示完整邏輯，並標註「Hooks Mainnet 上線後部署」。Hooks Testnet 已可驗證功能正確性，主網上線時零修改即可遷移。

Q：真實使用量能支撐 AMM 流動性嗎？ A：初期由平台作為做市商注入流動性，隨使用者增加自然形成雙邊市場。這是所有 DeFi 協議的標準起步路徑。

文件整理自規劃會議，涵蓋系統架構、XRPL 特性應用、代幣經濟、MVP 規劃與競賽論述。